



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name: Devansh Singh
PRN: 25070122072
Batch: A-2
Division: A

Subject: Programming with Java

ASSIGNMENT No : 27

TITLE : Develop a Java application demonstrating socket programming for communication between a client and a server.

Problem Statement

Develop a Java application demonstrating socket programming for communication between a client and a server.

Learning Objectives

To establish communication between client and server using Java sockets.

Theory

Socket programming enables communication between computers over a network using TCP/IP protocols. Java provides the `Socket` class for client-side communication and the `ServerSocket` class for server-side communication. A server listens for client requests on a specific port, while clients establish connections using the server's IP address and port number. Once connected, data is exchanged using input and output streams. Socket programming supports reliable communication between distributed systems and is widely used in chat applications, online gaming, file transfer systems, remote monitoring, and client-server architectures.

Output Screenshots

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp27/Main.java
Server waiting...
Client connected.
Client: Hello from Client
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp27/Client.java
Server: Hello from Server
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

CODE

```
1 package Exp27;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class Main {
7     public static void main(String[] args) {
8         try {
9             ServerSocket server = new ServerSocket(5000);
10            System.out.println("Server waiting...");
11
12            Socket socket = server.accept();
13            System.out.println("Client connected.");
14
15            BufferedReader in = new BufferedReader(
16                new InputStreamReader(socket.getInputStream()))
17            );
18
19            PrintWriter out = new PrintWriter(
20                socket.getOutputStream(), true
21            );
22
23            String message = in.readLine();
24            System.out.println("Client: " + message);
25
```

```

26         out.println("Hello from Server");
27
28         socket.close();
29         server.close();
30     }
31     catch (Exception e) {
32         System.out.println("Error: " + e.getMessage());
33     }
34 }
35 }

```

CLIENT CODE

```

1 package Exp27;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class Client {
7     public static void main(String[] args) {
8         try {
9             Socket socket = new Socket("localhost", 5000);
10
11             BufferedReader in = new BufferedReader(
12                 new InputStreamReader(socket.getInputStream())
13             );
14
15             PrintWriter out = new PrintWriter(
16                 socket.getOutputStream(), true
17             );
18
19             out.println("Hello from Client");
20
21             System.out.println("Server: " + in.readLine());
22
23             socket.close();
24         }
25         catch (Exception e) {
26             System.out.println("Error: " + e.getMessage());
27         }
28     }
29 }
30

```

EXERCISE 1

```
1 package Exp27.Exercise;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class Exp1 {
7     public static void main(String[] args) {
8         try {
9             ServerSocket server = new ServerSocket(5000);
10            System.out.println("Waiting for client...");
11
12            Socket socket = server.accept();
13
14            BufferedReader in = new BufferedReader(
15                new InputStreamReader(socket.getInputStream()))
16            );
17
18            PrintWriter out = new PrintWriter(
19                socket.getOutputStream(), true
20            );
21
22            System.out.println("Client: " + in.readLine());
23
24            out.println("Hello Client, message received.");
25
26            socket.close();
27            server.close();
28        }
29        catch (Exception e) {
30            System.out.println("Error: " + e.getMessage());
31        }
32    }
33 }
```

CLIENT CODE 1

```
1 package Exp27.Exercise;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class Client1 {
7     public static void main(String[] args) {
8         try {
9             Socket socket = new Socket("localhost", 5000);
10
11             BufferedReader in = new BufferedReader(
12                 new InputStreamReader(socket.getInputStream())
13             );
14
15             PrintWriter out = new PrintWriter(
16                 socket.getOutputStream(), true
17             );
18
19             out.println("Hello Server!");
20
21             System.out.println("Server: " + in.readLine());
22
23             socket.close();
24         }
25         catch (Exception e) {
26             System.out.println("Error: " + e.getMessage());
27         }
28     }
29 }
```

EXERCISE 2

```
1 package Exp27.Exercise;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class Exp2 {
7     public static void main(String[] args) {
8         try {
9             ServerSocket server = new ServerSocket(5000);
10            System.out.println("Waiting for client...");
11
12            Socket socket = server.accept();
13
14            BufferedReader in = new BufferedReader(
15                new InputStreamReader(socket.getInputStream()))
16            );
17
18            PrintWriter out = new PrintWriter(
19                socket.getOutputStream(), true
20            );
21
22            String message = in.readLine();
23
24            System.out.println("Client: " + message);
25
26            out.println(message);
27
28            socket.close();
29            server.close();
30        }
31        catch (Exception e) {
32            System.out.println("Error: " + e.getMessage());
33        }
34    }
35 }
```

CLIENT CODE 2

```
1 package Exp27.Exercise;
2
3 import java.io.*;
4 import java.net.*;
5
6 public class Client2 {
7     public static void main(String[] args) {
8         try {
9             Socket socket = new Socket("localhost", 5000);
10
11             BufferedReader in = new BufferedReader(
12                 new InputStreamReader(socket.getInputStream())
13             );
14
15             PrintWriter out = new PrintWriter(
16                 socket.getOutputStream(), true
17             );
18
19             out.println("Hello Server");
20
21             System.out.println("Echo from Server: " + in.readLine());
22
23             socket.close();
24         }
25         catch (Exception e) {
26             System.out.println("Error: " + e.getMessage());
27         }
28     }
29 }
```

Exercise

Q1. Create a simple client-server chat application using sockets.

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp27/Exercise/Exp1.java
Waiting for client...
Client: Hello Server!
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp27/Exercise/Client1.java
Server: Hello Client, message received.
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Q2. Create a Java program using socket programming where the client sends a message to the server, and the server returns the same message back (echo system).

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp27/Exercise/Exp2.java
Waiting for client...
Client: Hello Server
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp27/Exercise/Client2.java
Echo from Server: Hello Server
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Conclusion

Thus, the experiment successfully demonstrated socket programming in Java for communication between a client and a server. The `Socket` and `ServerSocket` classes were used to establish the connection. Input and output streams were used to exchange data between the client and server. The experiment provided an understanding of basic client-server communication using TCP/IP.